



Ariel Soto

Escuela de Administración y Negocios
Universidad de Concepción · Campus Chillán

SEMESTRE 1 · 2026



Fundamentos de Programación para Análisis Económico

MÓDULO II · SEMANA 5 · SESIÓN 2

group_by y Aplicaciones

La pregunta que dejamos abierta

Sesión 1: `summarise()` resume **todo** el data frame a una fila. Pero la pregunta económica casi nunca es del total...

...es **por grupo**: ingreso por sector, por región, por género. Ahí entra `group_by()`.

Al terminar esta sesión podrás:

- ▶ Calcular estadísticas **por grupo** con `group_by()` + `summarise()`
- ▶ Ordenar resultados con `arrange()`
- ▶ Combinar todos los verbos en un **workflow** completo y legible

💬 **Activación:** “¿gana lo mismo un trabajador de Servicios que uno de Agricultura?” Esa es una pregunta de `group_by`.

Bloque A — group_by()

`group_by()` cambia el significado de `summarise()`

`group_by()` no transforma los datos: les pone una **etiqueta de grupo**. El cálculo siguiente se hace **dentro de cada grupo**.

```
# Sin agrupar: una sola cifra (todo el dataset)
casen |> summarise(ingreso_medio = mean(ingreso, na.rm = TRUE))
```

```
  ingreso_medio
1      655290.9
```

```
# Agrupando por sector: una cifra POR sector
casen |>
  group_by(sector) |>
  summarise(ingreso_medio = mean(ingreso, na.rm = TRUE))
```

```
# A tibble: 6 × 2
  sector      ingreso_medio
  <chr>         <dbl>
1 Agricultura    447846.
2 Comercio      523833.
3 Construcción   520800
4 Educación     908857.
5 Industria     697889.
6 Servicios     788600
```

Mismo `summarise()`, resultado completamente distinto. `group_by()` es el interruptor “total ↔ por grupo”.

El patrón split-apply-combine

`group_by()` **divide** la tabla en grupos, `summarise()` **aplica** el cálculo a cada uno, y dplyr **combina** los resultados en una tabla nueva.

```
casen |>
  group_by(genero) |>
  summarise(ingreso_medio = mean(ingreso, na.rm = TRUE),
            n = n())          # n() = cuántos hay en cada grupo
```

```
# A tibble: 2 × 3
  genero ingreso_medio     n
  <chr>      <dbl> <int>
1 F          636296.    29
2 M          673607.    31
```

 **IMAGEN:** una tabla que se parte en grupos por color, cada grupo colapsa a una fila, y se recombinan.

Bloque B — Aplicaciones

Varios estadísticos a la vez

```
casen |>
  group_by(sector) |>
  summarise(
    n          = n(),
    ingreso_medio = mean(ingreso, na.rm = TRUE),
    ingreso_med   = median(ingreso, na.rm = TRUE),
    educ_media    = mean(educ)
  )
```

```
# A tibble: 6 × 5
  sector      n ingreso_medio ingreso_med educ_media
  <chr>    <int>         <dbl>         <dbl>    <dbl>
1 Agricultura    13      447846.      467000     12.3
2 Comercio       6      523833.      539500     11.3
3 Construcción   5       520800      502000      10
4 Educación       8      908857.      934000     16.2
5 Industria     11      697889.      694000     13.3
6 Servicios     17      788600      807000     12.4
```

Una sola consulta entrega un **perfil completo** por sector: tamaño, ingreso medio y mediano, educación. Esto es un análisis exploratorio de verdad.

Agrupar por más de una variable

```
casen |>
  group_by(region, genero) |>
  summarise(ingreso_medio = mean(ingreso, na.rm = TRUE),
            n = n(), .groups = "drop")
```

```
# A tibble: 10 × 4
  region      genero ingreso_medio     n
  <chr>      <chr>      <dbl> <int>
1 Araucanía  F          550800     6
2 Araucanía  M          726400     5
3 Biobío     F          592167     6
4 Biobío     M          694750     4
5 Maule      F          666200     5
6 Maule      M          653500     7
7 Metropolitana F          709750     4
8 Metropolitana M          596429     8
9 Ñuble      F          671857     8
10 Ñuble     M          725667     7
```

Con dos variables se obtiene una fila por **combinación** (región × género). El `.groups = "drop"` cierra la agrupación al terminar — buena práctica para evitar sorpresas en pasos siguientes.

Bloque C — arrange()

Ordenar los resultados

```
casen |>
  group_by(sector) |>
  summarise(ingreso_medio = mean(ingreso, na.rm = TRUE)) |>
  arrange(desc(ingreso_medio))      # de mayor a menor
```

```
# A tibble: 6 × 2
  sector      ingreso_medio
  <chr>      <dbl>
1 Educación  908857.
2 Servicios  788600
3 Industria  697889.
4 Comercio   523833.
5 Construcción 520800
6 Agricultura 447846.
```

`arrange()` ordena de menor a mayor; `desc()` invierte el orden. Ordenar el resultado es lo que convierte una tabla en un **ranking** legible.

Bloque D — Workflow completo

Todo junto: una pregunta, un pipe

“¿Cuál es el ingreso medio por sector entre trabajadores de 25–60 años, ordenado de mayor a menor?”

```
casen |>
  filter(!is.na(ingreso), edad >= 25, edad <= 60) |> # limpiar + acotar
  group_by(sector) |> # agrupar
  summarise(ingreso_medio = mean(ingreso), # resumir
            n = n()) |>
  arrange(desc(ingreso_medio)) # ordenar
```

```
# A tibble: 6 × 3
  sector      ingreso_medio     n
  <chr>          <dbl> <int>
1 Educación      874000     4
2 Servicios      811636.    11
3 Industria      730500     6
4 Construcción   532250     4
5 Agricultura    478333.     9
6 Comercio      447000     3
```

Cinco verbos, una pregunta, un flujo que se lee de arriba abajo como una frase. **Esto es análisis de datos profesional.**

Crear variables y luego agrupar

El flujo `mutate` → `group_by` → `summarise` aparece en TODA la T2:

```
casen |>
  mutate(grupo_etario = if_else(edad < 40, "joven", "mayor")) |>
  group_by(grupo_etario) |>
  summarise(ingreso_medio = mean(ingreso, na.rm = TRUE),
            n = n())
```

```
# A tibble: 2 × 3
  grupo_etario ingreso_medio     n
  <chr>         <dbl> <int>
1 joven         689682.     23
2 mayor         632364.     37
```

Primero se **construye** la variable de grupo (`mutate` + `if_else`), luego se **agrupa** por ella. Es el patrón para comparar tramos de edad, pobreza, género, etc.

Cierre: lo que nos llevamos esta semana

Pieza	Hace
<code>group_by()</code>	define los grupos del cálculo
<code>summarise()</code> + <code>n()</code>	una fila de resumen por grupo
<code>arrange()</code> / <code>desc()</code>	ordena → ranking
<code>filter \> mutate \> group_by \> summarise \> arrange</code>	el workflow completo
En una línea: dplyr responde preguntas económicas por grupo, de forma legible y reproducible.	
Al laboratorio (prepara la T2, calificada 13 %): limpieza y análisis de CASEN con dplyr — crear variables derivadas (pobreza, grupo etario), filtrar, resumir por sector/región/género y guardar el dataset limpio. Todo lo de esta semana se usa ahí.	



Sesión 2 — Fin

Al laboratorio práctico (2h)

T2 (calificada, 13 %) — Limpieza y análisis exploratorio de CASEN con dplyr `group_by` · `summarise` · variables derivadas · dataset limpio